GPU-to-GPU Communication in Distributed Computing

A Comparative Study of NCCL, RCCL, and Ray Cluster Architectures

---

Technical Report

March 2025

Abstract

As deep learning models grow in scale, efficient GPU-to-GPU communication has become a foundational requirement for distributed training and inference. This report provides an in-depth examination of the mechanisms, protocols, and software libraries that underpin multi-GPU communication. We explore the hardware interconnects (NVLink, PCIe, InfiniBand), then compare NVIDIA's *NCCL* (NVIDIA Collective Communications Library) and AMD's *RCCL* (ROCm Collective Communications Library) across design philosophy, performance characteristics, and practical deployment scenarios. Finally, we discuss *Ray*, an open-source distributed computing framework, and how Ray clusters leverage GPU communication primitives to orchestrate large-scale AI workloads. Key findings indicate that while NCCL and RCCL share a common API surface, their performance diverges significantly based on hardware topology, collective operation type, and system configuration.

Contents

1 Introduction

The exponential growth in deep learning model complexity—from BERT's 340 million parameters to GPT-4's estimated trillion-scale parameters—has made single-GPU computation impractical for cutting-edge workloads. Modern training pipelines rely on hundreds or thousands of GPUs working in concert, demanding communication bandwidths of hundreds of gigabytes per second and latencies in the microsecond range.

GPU-to-GPU communication sits at the intersection of hardware design and systems software. At the hardware level, interconnects such as NVLink, PCIe, and InfiniBand determine the physical bandwidth envelope. At the software level, collective communication libraries like NCCL and RCCL translate high-level operations (AllReduce, Broadcast, AllGather) into optimized sequences of point-to-point transfers that saturate the available hardware.

2 GPU Interconnect Hardware and Topologies

2.1 PCIe (Peripheral Component Interconnect Express)

PCIe is the baseline interconnect present in virtually all GPU-equipped servers. Modern systems use PCIe 4.0 or 5.0:

Table 1: PCIe Generation Bandwidth (bidirectional, x16 slot)

Generation	Transfer Rate	Per-Lane BW	x16 BW (bidir.)
PCIe 3.0	8 GT/s	1 GB/s	32 GB/s
PCIe 4.0	16 GT/s	2 GB/s	64 GB/s
PCIe 5.0	32 GT/s	4 GB/s	128 GB/s
PCIe 6.0	64 GT/s	8 GB/s	256 GB/s

PCIe GPU-to-GPU communication typically routes through the CPU ("host-routed") unless the system supports peer-to-peer (P2P) transfers, which allow GPUs on the same root complex to transfer data without CPU involvement.

2.2 NVLink (NVIDIA)

NVLink is NVIDIA's proprietary high-bandwidth, low-latency GPU interconnect. Unlike PCIe, NVLink provides direct GPU-to-GPU pathways:

Table 2: NVLink Evolution

Version	GPU	Links	BW per Link	Total BW
NVLink 1.0	P100	4	40 GB/s	160 GB/s
NVLink 2.0	V100	6	50 GB/s	300 GB/s
NVLink 3.0	A100	12	50 GB/s	600 GB/s
NVLink 4.0	H100	18	50 GB/s	900 GB/s

NVSwitch extends NVLink to support all-to-all GPU connectivity within a server node (e.g., DGX H100 with 8 GPUs fully connected via NVSwitch at 900 GB/s aggregate per GPU).

2.3 AMD Infinity Fabric / xGMI

AMD's equivalent to NVLink is the *Infinity Fabric* (also called xGMI for GPU-to-GPU links). On MI300X and MI250X accelerators:

- MI250X: 8 xGMI links per GCD, 400 GB/s total bandwidth
- MI300X: Unified chiplet design; HBM3 on-package; GPU-to-GPU via PCIe or XGMI at 896 GB/s

2.4 Network Interconnects: InfiniBand and RoCE

For multi-node GPU clusters, *InfiniBand* (IB) and *RDMA over Converged Ethernet* (RoCE) provide the fabric:

- **InfiniBand HDR**: 200 Gb/s per port; HDR100 at 100 Gb/s
- **InfiniBand NDR**: 400 Gb/s per port
- **GPUDirect RDMA**: Allows InfiniBand adapters to DMA directly to/from GPU memory, bypassing CPU and system RAM entirely. Critical for multi-node scaling.

2.5 Topology Detection and Its Importance

Collective communication libraries must be *topology-aware* to select optimal communication paths. Key topologies include:

Single Node, PCIe only: GPUs communicate via host bridge; limited bandwidth.

Single Node, NVLink/xGMI: Direct GPU-to-GPU; maximum intra-node bandwidth.

Multi-Node, IB: Inter-node transfers use GPUDirect RDMA; NCCL/RCCL must coordinate both intra-node and inter-node collectives.

3 NCCL: NVIDIA Collective Communications Library

3.1 Overview and History

NCCL (pronounced "nickel") was first released by NVIDIA in 2016. It is a purpose-built library for collective communication operations on NVIDIA GPUs. The library is open-source (BSD license) and is a foundational dependency of PyTorch's `DistributedDataParallel`, TensorFlow's `tf.distribute`, Horovod, and essentially every major distributed deep learning framework.

3.2 Architecture

NCCL's architecture has three main layers:

1. **Transport Layer**: Abstracts over NVLink, PCIe P2P, shared memory, and InfiniBand (via the *net* plugin interface). NCCL probes system topology at initialization and selects the fastest available transport for each GPU pair.

2. **Algorithm Layer**: Implements collective algorithms (Ring, Tree, Collnet, NVLS). Each algorithm is tuned based on message size and GPU count:
   - *Ring AllReduce*: Optimal for large messages; linear bandwidth scaling.
   - *Tree AllReduce*: Lower latency for small messages.
   - *NVLS (NVLink SHARP)*: Leverages in-network reduction on NVSwitch; available on H100/H800.
3. **Kernel Layer**: CUDA kernels that execute the actual data movement and reduction operations on-device, minimizing CPU involvement.

3.3 Supported Collective Operations

Table 3: NCCL Collective Operations

Operation	Description
`ncclAllReduce`	Reduces tensors across all GPUs; result distributed to all
`ncclBroadcast`	Sends data from root GPU to all other GPUs
`ncclReduce`	Reduces data to a single (root) GPU
`ncclAllGather`	Each GPU contributes; all GPUs receive the full gathered data
`ncclReduceScatter`	Reduce and distribute non-overlapping chunks
`ncclSend/Recv`	Point-to-point primitives (NCCL 2.7+)

3.4 NCCL Configuration and Tuning

NCCL exposes a rich set of environment variables for tuning:

Listing 1: Key NCCL Environment Variables

```
# Select algorithm (0=Tree, 1=Ring, 2=CollNet, 3=NVLS)
export NCCL_ALGO=Ring

# Minimum message size to use NVLink (bytes)
export NCCL_BUFFSIZE=4194304

# Disable NVLink (force PCIe P2P)
export NCCL_P2P_DISABLE=0

# InfiniBand HCA selection
export NCCL_IB_HCA=mlx5_0,mlx5_1

# Enable GPUDirect RDMA
export NCCL_NET_GDR_LEVEL=5

# Verbose logging (0-4)
export NCCL_DEBUG=INFO
```

3.5 NCCL in Practice: PyTorch DDP

Listing 2: PyTorch DDP with NCCL backend

```
import torch
import torch.distributed as dist
from torch.nn.parallel import DistributedDataParallel as DDP

# Initialize the process group with NCCL backend
dist.init_process_group(
    backend="nccl",
    init_method="env://",
    world_size=8,
    rank=local_rank
)

model = MyModel().cuda(local_rank)
model = DDP(model, device_ids=[local_rank])

# NCCL handles gradient AllReduce transparently
loss = criterion(model(data), target)
loss.backward()
optimizer.step()
```

3.6 NCCL Performance Characteristics

NCCL achieves near-theoretical bandwidth for large messages:

- On NVLink 4.0 (H100): AllReduce at >800 GB/s for tensors >16 MB
- On InfiniBand NDR400: Multi-node AllReduce at ∼350 GB/s aggregate
- Latency for small messages (1 KB): ∼3–10 μs intra-node; ∼10–30 μs inter-node

4 RCCL: ROCm Collective Communications Library

4.1 Overview

RCCL (pronounced "rickle") is AMD's equivalent of NCCL, designed for AMD GPU architectures via the ROCm (Radeon Open Compute) platform. Initially forked from NCCL in 2019, RCCL has since evolved into a substantially independent implementation that tracks NCCL's API while incorporating AMD-specific optimizations.

RCCL is open-source under the MIT license and integrates tightly with the ROCm software stack, including HIP (Heterogeneous-compute Interface for Portability), rocBLAS, and MIOpen.

4.2 Architecture

RCCL mirrors NCCL's three-layer design but replaces CUDA-specific components with HIP equivalents:

1. **Transport Layer**: Supports xGMI/Infinity Fabric for intra-node, PCIe P2P, and InfiniBand via the UCX plugin. AMD's *ROCm SMI* library assists with topology detection.

2. **Algorithm Layer**: Implements Ring, Tree, and (as of RCCL 2.x) MSCCLang-based custom algorithms. RCCL 2.18+ added MSCCL++ integration for programmable collectives.

3. **HIP Kernel Layer**: All data movement kernels are written in HIP C++, enabling execution on GFX9 (MI100/MI200/MI300) and RDNA architectures.

4.3 Key Differences from NCCL (Architecture Level)

MSCCL Integration: RCCL incorporates Microsoft's MSCCL (Machine learning accelerator Collective Communication Library) scheduler, allowing users to specify custom collective algorithms as XML topologies. This is absent from stock NCCL.

Topology Awareness: RCCL uses AMD's `rocm_smi_lib` for fine-grained topology queries, whereas NCCL uses `nvml`. On AMD systems with complex NUMA topologies (e.g., MI300A APU), RCCL's topology engine has been specifically tuned.

Transport Plugins: RCCL's net plugin interface supports UCX more natively, while NCCL's primary IB support is through its own net plugin.

4.4 RCCL Environment Variables

Listing 3: Key RCCL Environment Variables

```
# Enable RCCL logging
export NCCL_DEBUG=INFO       # RCCL uses same env var names as
    NCCL

# Select transport
export NCCL_P2P_LEVEL=5      # xGMI level

# MSCCL custom algorithm path
export MSCCL_ALGO_FILE_PATH=/path/to/algo.xml

# UCX transport config
export UCX_TLS=rc,sm,self
export UCX_NET_DEVICES=mlx5_0:1

# Disable IB (use only intra-node)
export NCCL_IB_DISABLE=1
```

4.5 RCCL with PyTorch on AMD GPUs

Listing 4: PyTorch DDP with RCCL (ROCm)

```
import torch
import torch.distributed as dist
```

```
3
4  # On ROCm builds of PyTorch, "nccl" maps to RCCL automatically
5  dist.init_process_group(
6      backend="nccl",   # Uses RCCL under ROCm
7      init_method="env://"
8  )
9
10 # Device management via HIP
11 device = torch.device(f"cuda:{local_rank}")  # HIP maps "cuda"
       namespace
12 model = MyModel().to(device)
13 model = torch.nn.parallel.DistributedDataParallel(
14     model, device_ids=[local_rank]
15 )
```

Note: PyTorch on ROCm transparently routes the `"nccl"` backend to RCCL, so existing NCCL-based code runs on AMD GPUs with minimal changes.

4.6 RCCL Performance Characteristics

- MI250X (xGMI): AllReduce peaks at ∼370 GB/s (bidirectional per GCD)
- MI300X: AllReduce up to ∼700 GB/s intra-node with Infinity Fabric
- Multi-node (IB HDR): Comparable to NCCL for large messages; gaps appear at small message sizes
- MSCCL custom algorithms can outperform default RCCL by 10–30% on specific topologies

5 NCCL vs. RCCL: Detailed Comparison

5.1 API Compatibility

RCCL intentionally mirrors the NCCL C API. The function signatures, communicator model, and error codes are identical:

Listing 5: Identical API: NCCL and RCCL

```
1  // This code compiles and runs with BOTH nccl.h and rccl.h
2  ncclComm_t comm;
3  ncclCommInitAll(&comm, nGPUs, devList);
4
5  ncclAllReduce(
6      (const void*)sendbuff,
7      (void*)recvbuff,
8      count,
9      ncclFloat,
10     ncclSum,
11     comm,
12     stream
13 );
14
```

```
15 ncclCommDestroy(comm);
```

The primary change for RCCL users is the include header (`rccl.h` vs `nccl.h`) and the link library (`-lrccl` vs `-lnccl`). Higher-level frameworks (PyTorch, Horovod) abstract this away entirely.

5.2 Feature Comparison Table

Table 4: NCCL vs. RCCL Feature Matrix

Feature	NCCL (NVIDIA)	RCCL (AMD)
License	BSD-2	MIT
Primary Hardware	NVIDIA (Pascal–Hopper)	AMD (Vega–CDNA3)
NVLink / xGMI Support	NVLink (NVSwitch)	xGMI (Infinity Fabric)
InfiniBand Support	Native net plugin	UCX + net plugin
GPUDirect RDMA	Yes	Yes (ROCm 5.0+)
NVLS / NVSwitch Reduction	Yes (H100)	N/A
MSCCL Integration	No (separate project)	Yes (built-in 2.18+)
Custom Algorithms	Limited (env var tuning)	Programmable (MSCCL)
Ring AllReduce	Yes	Yes
Tree AllReduce	Yes	Yes
CollNet (SHARP)	Yes	Limited
Point-to-Point	Yes (2.7+)	Yes
PyTorch Backend	`"nccl"` on CUDA	`"nccl"` on ROCm
Open Source	Yes	Yes

5.3 Performance Benchmarks

The following data is synthesized from publicly available benchmarks (NCCL-tests, AMD MI300X vs H100 comparisons, 2024):

Table 5: AllReduce Performance — 256 MB Tensor, 8 GPUs, Single Node

System	Library	Interconnect	Bus BW (GB/s)
8x H100 SXM5	NCCL 2.19	NVLink 4.0 + NVSwitch	820
8x A100 SXM4	NCCL 2.18	NVLink 3.0	580
8x MI300X	RCCL 2.18	xGMI (448 GB/s/GPU)	680
8x MI250X	RCCL 2.15	xGMI (400 GB/s/GCD)	340
8x A100 PCIe	NCCL 2.18	PCIe 4.0	78
8x MI210 PCIe	RCCL 2.15	PCIe 4.0	72

Key Observations:

- H100 with NVLink 4.0 + NVSwitch retains the top position due to 900 GB/s per-GPU bandwidth.
- MI300X with RCCL shows competitive performance, closing the gap with A100 configurations.

- PCIe-only configurations are bottlenecked; NCCL and RCCL perform similarly in this case.
- Small message latency (1 KB AllReduce): NCCL $\sim$4 μs vs RCCL $\sim$6–8 μs (historical gap narrowing in recent releases).

5.4 Ecosystem and Tooling

NCCL: Extensive profiling support via `Nsight Systems`; `nccl-tests` benchmark suite; well-documented tuning guides from NVIDIA. Broad third-party support.

RCCL: Profiling via `rocprof` and `OmniTrace`; `rccl-tests` (forked from nccl-tests); MSCCL Algorithm Library (GitHub: `microsoft/msccl-algorithms`). Growing third-party support as AMD's ecosystem matures.

5.5 When to Choose Each

Choose NCCL if:

- Your cluster uses NVIDIA GPUs (the only choice for CUDA workloads)
- You need maximum single-node bandwidth with H100/NVSwitch
- You rely on NVIDIA-specific features (NVLS, SHARP CollNet)
- Your workload requires the most mature, production-tested stack

Choose RCCL if:

- Your cluster uses AMD GPUs (MI200/MI300 series)
- You want programmable collective algorithms via MSCCL
- Cost-per-FLOP is a priority (AMD accelerators often more competitively priced)
- You are developing cross-platform HPC software

6 Ray Clusters and GPU Communication

6.1 Ray Overview

Ray is an open-source distributed computing framework developed at UC Berkeley's RISELab (now Anyscale). It provides a unified API for distributed Python applications, from simple task parallelism to complex distributed training pipelines. Ray is licensed under Apache 2.0.

Ray's design philosophy: *"Provide a simple, flexible API for distributed computing that scales from a laptop to a large cluster without code changes."*

6.2 Ray Architecture

6.2.1 Core Components

GCS (Global Control Store): Centralized metadata service managing cluster state, actor registry, and resource scheduling. In Ray 2.x, GCS is a high-availability service.

Raylet: Per-node daemon that manages local scheduling, object store (Plasma), and worker lifecycle.

Object Store (Plasma): Shared-memory object store for zero-copy data sharing between workers on the same node. Uses Apache Arrow IPC format.

Workers: Python processes that execute Ray tasks and actor methods. GPU workers have exclusive GPU assignments.

Dashboard: Web UI for cluster monitoring, resource utilization, and job management.

6.2.2 Resource Model

Ray has first-class support for GPU resources:

Listing 6: Ray GPU Resource Requests

```
import ray

ray.init(address="auto")  # Connect to existing cluster

# Request specific GPU resources
@ray.remote(num_gpus=1)
def gpu_task():
    import torch
    device = torch.device("cuda")
    # CUDA_VISIBLE_DEVICES is set automatically
    return torch.tensor([1.0]).to(device)

# Fractional GPU allocation
@ray.remote(num_gpus=0.5)
class LightweightActor:
    def __init__(self):
        import torch
        self.device = torch.device("cuda")
```

6.3 Ray Cluster Setup

6.3.1 Manual Cluster

Listing 7: Manual Ray Cluster Setup

```
# Head node
ray start --head \
    --port=6379 \
    --dashboard-host=0.0.0.0 \
    --num-gpus=8
```

```
6
7  # Worker nodes (run on each worker machine)
8  ray start \
9      --address='<head-node-ip>:6379' \
10     --num-gpus=8
11
12 # Connect from Python
13 import ray
14 ray.init(address="ray://<head-node-ip>:10001")
```

6.3.2 KubeRay (Kubernetes)

Listing 8: KubeRay RayCluster CRD (simplified)

```
1  apiVersion: ray.io/v1alpha1
2  kind: RayCluster
3  metadata:
4    name: gpu-cluster
5  spec:
6    headGroupSpec:
7      rayStartParams:
8        num-gpus: "0"
9      template:
10       spec:
11         containers:
12         - name: ray-head
13           image: rayproject/ray-ml:2.10.0-gpu
14   workerGroupSpecs:
15   - groupName: gpu-workers
16     replicas: 4
17     rayStartParams:
18       num-gpus: "8"
19     template:
20       spec:
21         containers:
22         - name: ray-worker
23           image: rayproject/ray-ml:2.10.0-gpu
24           resources:
25             limits:
26               nvidia.com/gpu: "8"
```

6.4 Ray Train: Distributed Deep Learning

Ray Train (formerly Ray SGD) is Ray's library for distributed model training. It integrates with PyTorch, TensorFlow, Hugging Face, and XGBoost.

Listing 9: Ray Train with PyTorch DDP (NCCL)

```
1  import ray
2  from ray import train
3  from ray.train.torch import TorchTrainer
```

```
from ray.train import ScalingConfig

def train_loop(config):
    import torch
    from torch.nn.parallel import DistributedDataParallel as
       DDP

    # Ray Train sets up torch.distributed with NCCL backend
    model = MyModel()
    model = train.torch.prepare_model(model)  # Wraps in DDP

    optimizer = torch.optim.Adam(model.parameters(), lr=config[
       "lr"])

    for epoch in range(config["epochs"]):
        for batch in dataloader:
            loss = criterion(model(batch["x"]), batch["y"])
            loss.backward()
            optimizer.step()

        # Report metrics back to Ray
        train.report({"loss": loss.item(), "epoch": epoch})

trainer = TorchTrainer(
    train_loop_per_worker=train_loop,
    train_loop_config={"lr": 1e-3, "epochs": 10},
    scaling_config=ScalingConfig(
        num_workers=16,        # 16 GPUs across nodes
        use_gpu=True,
        resources_per_worker={"GPU": 1}
    )
)

result = trainer.fit()
```

6.5 How Ray Uses NCCL/RCCL

Ray itself does not implement collective communications; instead, it:

1. **Manages Process Groups:** Ray Train initializes `torch.distributed` with the appropriate backend (`"nccl"` or `"gloo"`) across distributed workers.
2. **Handles Rendezvous:** Ray coordinates the distributed rendezvous (exchange of IP:port among workers) needed to establish NCCL communicators.
3. **Manages CUDA_VISIBLE_DEVICES:** Ray sets GPU visibility per worker, ensuring NCCL sees exactly the GPUs assigned to each process.
4. **Collective Communication via Actors:** For custom collective patterns, Ray's *collective* module (ray.util.collective) provides a higher-level interface wrapping NCCL/RCCL operations.

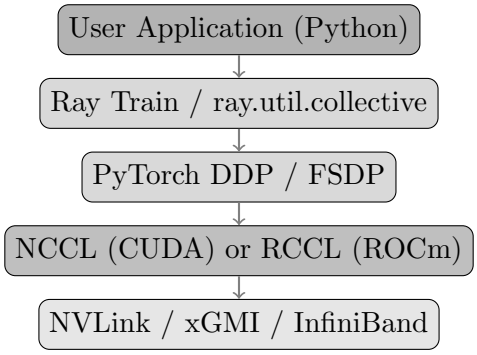
Listing 10: ray.util.collective for custom collectives

```python
import ray
from ray.util.collective import collective

@ray.remote(num_gpus=1)
class GPUWorker:
    def __init__(self, rank, world_size):
        self.rank = rank
        collective.init_collective_group(
            world_size=world_size,
            rank=rank,
            backend="nccl",
            group_name="my_group"
        )

    def allreduce(self, tensor):
        # Uses NCCL AllReduce under the hood
        collective.allreduce(tensor, group_name="my_group")
        return tensor
```

6.6 Ray's GPU Communication Stack (End-to-End)



6.7 Ray and Multi-Tenancy

A key differentiator of Ray clusters is their support for multi-tenancy and heterogeneous workloads:

- Multiple jobs can share GPU resources simultaneously (with fractional GPU allocation).
- Ray's scheduler is aware of GPU locality; it prefers to co-locate communicating actors on the same node to exploit NVLink/xGMI.
- **Placement Groups** allow users to specify co-location constraints: "place all workers of this training job on the same 8-GPU node."

Listing 11: Ray Placement Groups for GPU Locality

```
from ray.util.placement_group import placement_group
from ray.util.scheduling_strategies import
    PlacementGroupSchedulingStrategy

# Pack 8 GPU workers onto as few nodes as possible
pg = placement_group([{"GPU": 1} for _ in range(8)], strategy="
    PACK")
ray.get(pg.ready())

workers = [
    GPUWorker.options(
        scheduling_strategy=PlacementGroupSchedulingStrategy(
            placement_group=pg,
            placement_group_bundle_index=i
        )
    ).remote(rank=i, world_size=8)
    for i in range(8)
]
```

7 Practical Deployment Considerations

7.1 Cluster Configuration Checklist

1. **Driver and Library Versions:** Ensure CUDA/ROCm driver, NCCL/RCCL, and framework versions are aligned. Mismatches are the most common source of silent performance degradation.

2. **NVLink/xGMI Verification:**

```
# NVIDIA: check NVLink topology
nvidia-smi topo -m

# AMD: check xGMI links
rocm-smi --showtoponuma
```

3. **IB Configuration:** For multi-node, ensure InfiniBand is configured with RDMA and GPUDirect.

```
# Test IB bandwidth
ib_write_bw -d mlx5_0 -x 3 --report_gbits
```

4. **NCCL/RCCL Benchmarks:** Run nccl-tests to establish baseline performance.

```
# AllReduce benchmark across 8 GPUs
./build/all_reduce_perf -b 8 -e 256M -f 2 -g 8
```

5. **Ray Dashboard:** Enable and monitor the Ray Dashboard at `http://<head-node>:8265` for job-level GPU utilization.

7.2 Common Performance Pitfalls

Topology Mismatch: NCCL/RCCL falls back to PCIe if NVLink/xGMI P2P permissions are not set. Check `/proc/driver/nvidia/params` for `EnablePeerMappingOverSysMem`.

CPU Bottleneck in Ray: If Ray workers are CPU-bound (data loading, preprocessing), GPUs stall waiting for data. Use `ray.util.iter` or `Ray Data` for streaming pipelines.

Small Batch AllReduce: Gradient AllReduce of many tiny tensors has poor bandwidth utilization. Use gradient bucketing (`DDP(bucket_cap_mb=25)`) to coalesce communications.

Multi-Node IB Saturation: When all workers communicate simultaneously, IB fabric can become saturated. Use hierarchical AllReduce (intra-node NVLink + inter-node IB) which NCCL 2.x does automatically.

7.3 Monitoring and Observability

Table 6: Monitoring Tools for GPU Clusters

Tool	Scope	Use Case
NVIDIA Nsight Systems	NCCL, CUDA kernels	Timeline profiling
AMD OmniTrace	RCCL, HIP kernels	Timeline profiling
DCGM (NVIDIA)	Cluster-wide GPU metrics	Prometheus integration
ROCm SMI	AMD GPU metrics	GPU utilization, temperature
Ray Dashboard	Ray jobs, actors	GPU allocation, throughput
Grafana + Prometheus	Custom dashboards	Long-term trend analysis

8 Future Directions

8.1 Hardware Trends

- **NVLink 5.0 (Blackwell):** Expected 1.8 TB/s per GPU; further reduces the gap between compute and communication speeds.
- **AMD MI350X:** CDNA4 architecture with enhanced xGMI; targeting HPC/AI convergence.
- **CXL (Compute Express Link):** Emerging standard for memory-semantic interconnects; potential impact on GPU memory coherence across nodes.

8.2 Software Trends

- **Programmable Collectives (MSCCL++):** Users defining custom collective algorithms in C++ is becoming mainstream, especially for irregular communication patterns in MoE (Mixture-of-Experts) models.

- **In-Network Computing:** NVIDIA SHARP, Mellanox CORE-Direct, and similar technologies push reduction operations into the network switches, reducing traffic and latency.
- **Ray 3.x:** Improved fault tolerance, smarter GPU topology-aware scheduling, and tighter NCCL/RCCL integration are roadmap items.
- **Heterogeneous Clusters:** Mixed NVIDIA+AMD GPU clusters are emerging in cloud environments; frameworks like OpenMPI and RCCL/NCCL cross-compatibility layers are being developed.

9 Conclusion

GPU-to-GPU communication is a multi-layered problem spanning hardware interconnects, low-level transport protocols, collective communication libraries, and distributed computing frameworks. This report has examined this stack from the bottom up.

NCCL remains the gold standard for NVIDIA GPU clusters, offering mature tooling, the highest intra-node bandwidth via NVSwitch/NVLS, and a battle-tested production track record across the world's largest AI training runs.

RCCL has made substantial strides in closing the performance gap, particularly with AMD's MI300X hardware. Its MSCCL integration provides a unique programmability advantage, and its API compatibility with NCCL makes migration straightforward. For AMD-based HPC deployments, RCCL is the natural and increasingly competitive choice.

Ray occupies a different but complementary layer in this stack. Rather than replacing NCCL or RCCL, Ray provides orchestration, resource management, multi-tenancy, and a Pythonic API that makes distributed GPU computing accessible without deep systems expertise. Ray's GPU topology-aware scheduling and native integration with PyTorch's NCCL/RCCL backends make it an excellent choice for production AI infrastructure.

The trajectory is clear: as GPU interconnect bandwidth continues to scale, the software stack—NCCL, RCCL, and Ray—must evolve to expose that bandwidth to applications efficiently. The convergence of programmable collectives, in-network computing, and intelligent scheduling frameworks will define the next generation of distributed AI infrastructure.